The time to engage with these technologies is now. Whether by integrating them into research workflows, critically evaluating their outputs, or contributing to their development, IR researchers must take an active role in shaping the future of AI-driven science. These systems are poised to transform how knowledge is produced and disseminated, and the IR community is uniquely positioned to lead this transformation.

2 AI Scientist: Functionality and Evaluation

This section includes an overview of the features and architecture of the AI Scientist, case study snippets, critiques and recommendations for use.

2.1 Setup and Installation

All setup, coding and experiment execution were carried out by the third author of this manuscript, a third-year computer science Bachelor's student with basic machine learning knowledge but strong Python proficiency. A typical user of the AI Scientist is likely to have greater expertise, such as that of a trained researcher or machine learning engineer. Thus, our setup time and usability assessments should be considered lower bounds. We have made our experimental codebase that replicates our experimentation in this paper available on GitHub: https://code.isg.beel.org/ISG-Siegen/the-ai-scientist-reproduced. The repository includes a Singularity container definition file to facilitate full reproducibility and modifications in our experiments.

We installed the AI Scientist on two computers and found the setup to be relatively straightforward, contrary to some reports on the Web [11]. The entire process took approximately five hours, including minor troubleshooting. For more details, we refer the interested reader to our GitHub repository.

We first tested and prototyped the AI Scientist on a consumer laptop[12]. The main experiments were then conducted on our university's computing cluster[13]. Although the AI Scientist provides an experimental pipeline optimized for deep learning with PyTorch and an NVIDIA GPU (CUDA), our experiments used a simple recommendation algorithm, FunkSVD, that runs on a CPU.

The AI Scientist requires a foundation large language model (LLM) to function. At the time of our experiments, it supported OpenAI's `gpt-4o-2024-05-13`, multiple models from Anthropic's `Claude` series, DeepSeek Coder V2, and Meta's `Llama 3.1`. We opted for `gpt-4o-2024-05-13`. Since then, Sakana has expanded support to additional models.

2.2 Preparing for the experiments

We initially assumed the AI Scientist could autonomously conduct research based solely on a prompt. However, it requires a user-defined "template," which significantly limits the autonomy of the AI Scientist. Such a template consist of several elements, described in the following.

1. Goal and Research Direction. The research goal is specified via a prompt file (*prompts.json*). For our experiments, we focused on Green Recommender Systems [3], a research topic which aims to reduce the carbon footprint of recommender systems and its associated machine learning algorithms. We gave the prompt as: [14]:

Listing 1. The general task provided as a JSON prompt

[11] https://www.reddit.com/r/learnmachinelearning/comments/1fuw2yb/has_anybody_gotten_sakana_ai_scientist_to_work/
[12] HUAWEI MateBook D 16, Windows 11, Intel Core i5-12450H with integrated graphics, 16GB RAM (3733 MT/s)
[13] Single node with 2 AMD EPYC 7452 CPUs (32 cores, 2.35-3.35 GHz, 128 MB cache) and 256 GB DDR4 RAM (3200 MHz), running Rocky Linux 8.8
[14] All prompts are shortened and rephrased for brevity; the originals are available in our GitHub repository.

```
{
  "task_description": "The attached code trains and evaluates the FunkSVD algorithm with stochastic
      ↪ gradient descent for recommender systems. Your goal is to find novel ways to optimize energy-
      ↪ efficiency. "
}
```

2. Experimental Pipeline. The AI Scientist requires a Python-based experimental pipeline, consisting of an `experiment.py`, which defines dataset usage and model training as well as a `plot.py`, which handles result visualization. We implemented FunkSVD for collaborative filtering, training and evaluating it on an 80/10/10 split on MovieLens-100k (RMSE metric), with results visualized via line charts. This minimalist setup runs on a CPU, but larger datasets and complex models would likely increase compute costs. It is important to note that the AI Scientist requires this pipeline in a special format. Users cannot simply input any Python code but would have to adjust their existing code to be able to be used by the AI Scientist.

3. Seed Ideas. The AI Scientist further requires user-provided seed ideas. We supplied two:

(1) **Adaptive Learning Rates for SGD.** Aims to accelerate convergence, reducing computational cost and energy use. Though not novel, this ideas demonstrates our intended research direction.
(2) **E-Fold Cross-Validation.** An alternative to k-fold cross-validation that dynamically selects an optimal e to balance computational efficiency and reliability. This idea is also not entirely novel, but was proposed only recently[1, 4, 9].

The seed ideas must include values for "interestingness," "feasibility," and "novelty." However, we observed that these values had no apparent impact on the AI Scientist's processing.

Listing 2. e-fold Cross Validation; one of the two seed ideas

```
{
    "Name": "e_fold_cv",
    "Title": "E-Fold Cross-Validation for Model Performance Estimation",
    "Experiment": "Develop an alternative to k-fold cross-validation named e-fold cross-validation. Instead
        ↪  of a static k it uses an intelligently chosen or dynamically adjusted paramter e to optimize
        ↪ the number of folds and to minimize computational energy while maintaining reliable model
        ↪ performance estimates.",
    "Interestingness": 8,
    "Feasibility": 7,
    "Novelty": 7
  }
```

4. LaTeX Templates. The AI Scientist generates research papers using predefined LaTeX templates. We used the default templates provided by Sakana without modification.

2.3 Idea Generation

Once provided with required input, the AI Scientist generated the following ten research ideas:

1. Factor pruning for energy-efficient matrix factorization
2. Dynamic factor adjustment in matrix factorization
3. Green metrics for sustainable recommender systems
4. Quantization techniques for energy-efficient SGD
5. Time-aware early stopping
6. Interaction-priority SGD for energy efficiency
7. Micro-batching for energy-efficient SGD
8. Sparse-aware SGD for matrix factorization
9. Cluster-aware SGD for matrix factorization
10. Hybrid matrix factorization

Each generated idea includes a brief description and three AI Scientist judged numerical scores (1–10) for Interestingness, Feasibility, and Novelty, along with a binary "novel" label (True/False). However, while the *novel* label affects its workflow, the other scores appear arbitrarily assigned and are not used in further processing. To determine novelty, the AI Scientist queries the Semantic Scholar API, retrieving up to 10 search results per query. The system extracts search result metadata (title, authors, venue, year, citations, and abstracts) and can iterate up to 10 times before deciding. If no clear matches are found, it assigns `novel=True`; otherwise, it may classify the idea as non-novel.

Despite mimicking a literature review, this approach is unreliable. In our experiments, the AI Scientist classified all 10 generated ideas and both seed ideas as novel[15], despite some being well-documented. For instance, micro-batching for SGD (Idea 7) is a known technique [6], as are adaptive learning rates (seed idea), hybrid matrix factorization (idea 10) and e-fold cross validation (seed idea). This misclassification likely results from the AI Scientist's reliance on keyword matching rather than any deeper synthesis of research. While some ideas may be novel within recommender systems, many simply apply existing techniques (e.g., pruning) to matrix factorization, offering incremental rather than groundbreaking improvements.

2.4 Conducting Experiments

The AI Scientist executes research ideas by modifying the user-provided experimental pipeline using `Aider`[16], an LLM-driven coding assistant that iterates on `experiment.py` up to five times. This reliance on `Aider` adds complexity, as errors in one system propagate to the other. While the AI Scientist does introduce variation, it remains highly dependent on the provided framework, reworking the predefined templates rather than devising novel methodologies.

Our code analysis suggests minimal changes per iteration. Our template had 6,260 characters of code (255 lines), with `Aider` adding, on average, 529 characters (+8%) in the first iteration, followed by 118, 83, 66, and 21 characters in subsequent iterations (see Table 1).

The AI Scientist struggled with both implementation and methodological correctness. It failed to execute five of twelve ideas due to unresolved coding errors, often cycling through flawed versions of the same code. Even when producing executable code, outputs were frequently unreliable. For instance, in our e-fold cross-validation experiment, the AI Scientist intended to test $e = \{2, 3, 4, 5\}$ but erroneously kept e fixed at 2, invalidating results. It also failed to re-run the baseline (k-fold), making e-fold cross validation appear superior – an impossible outcome because e-fold cross-validation by design cannot lead to better performance than k-fold cross validation but only energy savings (with,

[15] In prior tests, the AI Scientist occasionally flagged some ideas as non-novel, showing that it does not always default to a "novel" classification.
[16] https://Aider.chat/